

CMPUT275—Assignment 3 (Winter 2026)

R. Hackman

Due Date: Friday April 10th, 8:00PM

Per course policy you are allowed to engage in *reasonable* collaboration with your classmates. You must include in the comments of your assignment solutions a list of any students you collaborated with on that particular question.

In this assignment and all following assignments you should test against the sample executables. The sample executables try to help you out and print error messages when receiving invalid input (though they do not catch *all* invalid input). You do not have to replicate any error messages printed, unless the assignment specification specifically asks for error messages. These messages are only in the sample executable to help you catch when you write an invalid test case.

Memory Management: In order to complete some of these questions you will be required to use dynamic memory allocation. Your programs must not leak any memory, if you leak memory on a test case then you are considered to have failed that test case. You can test your program for memory leaks by using the tool `valgrind`.

Efficiency Requirements: While there are not strict efficiency guidelines, your code should strive to not be unnecessarily inefficient. If your solution is not comparable for large test cases in how long it takes to run compared to the sample executable then you may lose marks due to your program hitting the time limit. As such, it is good to compare your program to the sample executable not only for output but also timing purposes. The sample executable purposefully not overly optimized, so this should be an easy target to reach.

Compilation Flags: each of your programs should be compiled with the following command:

```
gcc -Wvla -Wall -Werror
```

These are the flags we'll compile your program with, and should they result in a compilation error then your code will not be compiled and ran for testing.

Allowed libraries: `stdio.h`, `stdlib.h`, `string.h`, and `assert.h`. No other libraries are allowed.

1. Linear Sequences

In this question you will be writing a C program `sequences.c` that will be able to compose arithmetic operators to form a linear sequence, and be asked to print out the next item in the sequence at any time. Your program will read in pairs of operators and operands which will make up the series of arithmetic operations to apply to each integer in the sequence to form the next integer in the sequence. Your program will have the following specifications for its arguments and its input.

- Your program will expect one command line argument which is an integer, representing the initial value of the sequence.
- Your program will then read input from standard input until reaching EOF, correctly formed input will always be one of the following
 - One of the operator strings `add`, `sub`, `mul`, or `div` followed by one integer.
 - Or, the string `n`
- When receiving one of the operator strings, your program will add that operator to the series of operators to apply to each number in the sequence to produce the next. The integer provided after each operator string is the right-hand operand to be used with that operator when applying it, the left-hand operand of each operator will always be the current integer in your series, or the result of applying a previous operation. When calculating the next integer in the series the operators must always be applied in the order they were received in input!
- When receiving the string `n` your program will calculate the next integer in the series, as currently defined by your operators, and print it out followed by a newline.

So, for example, if your command line argument was 3 and your program receives the following input

```
add 1 mul 2 n n
```

then your program would print out

```
8
18
```

because $(3 + 1) * 2 = 8$, and $(8 + 1) * 2 = 18$.

Deliverables For this question include in your final submission zip your C source code file named `sequences.c`

2. Maze Game

In this question, you will implement an ADT meant to represent a maze. For this question you have already been provided with the files `main.c` and `maze.h`. **You may not modify either of these files.** Even if you do modify those files and submit them they will be overwritten with the original files and if your solution required your modifications then your solution would no longer work, so do not modify them. The file `main.c` is the client code that uses your ADT, and the file `maze.h` is the header files that declares the functions that `main.c` relies on. Your job is to implement the ADT in the file `maze.c`.

Your maze will be represented by a two-dimensional grid of tiles, where each tile is represented by a character. The following are the types of tiles that comprise your maze:

- Start Tile - Denoted by the character 'S', this denotes where the player will begin in the maze when it is first created, or when it is reset. Every maze must have *exactly* one start tile. Otherwise, the start tile behaves exactly as an *Open Tile* described below.
- Goal Tile - Denoted by the character 'G', this denotes the “goal” or exit of the maze. When the player reaches this location its task is done. Each maze must have *at least* one goal tile.
- Open Tile - Denoted by the character 'O', this denotes a normal tile of the maze that the player is allowed to walk through.
- Wall Tile - Denoted by the character 'X', this denotes a solid wall tile of the maze that the player is not able to walk through.
- Teleporter Tiles - Denoted by any digit character ('0' through '9'), this denotes a teleporter tile. When the player walks on a teleporter tile their location becomes that of the other matching teleporter tile. You may assume that if a maze has a teleporter tile for a number there will be exactly two of that number appearing in the maze. If you are unsure about the behaviour, as always, test with the sample executable.
- Icy Tiles - Denoted by the character 'I', this denotes an open tile which the player may pass through but with an icy floor, so the player will continue sliding in the direction they were moving until hitting a wall, hitting the edge of the maze, or landing on the next non-icy tile. If you are unsure about the behaviour, as always, test with the sample executable.

In order to implement this ADT you must define the structure type `Maze` and all the functions declared in the provided header file `maze.h`. Each of the functions have comments describing them in the header file, but are also described here. The functions are as follows:

- `struct Maze *readMaze()` - This function read a maze in from standard input and returns a pointer to a heap-allocated `Maze` structure which you have initialized to represent the maze that was read in. When reading in a maze your function will read in lines of characters until reading in a completely blank line. Each line represents a row of the grid that makes your maze. You may assume EOF is not reached before completing the reading in of the `Maze`, however your function must make sure that it receives only exactly one start tile and at least one goal tile. If your function does not detect exactly one start tile and at least one goal tile then it instead returns a `NULL` pointer.
- `struct Pos makeMove(struct Maze *, char)` - This function takes a pointer to a valid `Maze` structure and a character that must be one of 'n', 'e', 's', or 'w' which represents the directions north, south, east, and west respectively. This attempts to move the player in the given direction, following the rules of the maze, and returns a `Pos` structure that stores the players new x and y coordinates as well as mutates the `Maze` structure so that the player has made the given move. If the players new position is the goal state the function instead returns a `Pos` structure with x and y coordinates both set to -1.

- `void reset(struct Maze *)` - This function *resets* the maze, which simply resets the player's current position to the original starting position of that maze.
- `void printMaze(struct Maze *)` - This function prints out the maze to the screen. This prints a border around the maze where the top and bottom are enclosed in '=' characters and the sides are enclosed in '|' characters. All the tiles of the maze are printed, except for the tile on which the player currently resides where instead the character 'P' is printed to convey that the player is located there.
- `struct Maze *destroyMaze(struct Maze *)` - This function frees all memory associated with the given `Maze` structure which was created with `readMaze`, including the `Maze` itself. Returns `NULL`.

Note: All of these functions work on a `Maze` structure which *you* get to define. Define the `Maze` structure in your `maze.c` file and make sure to include all the fields you will need to make the ADT behave as described.

Building and Testing: As you are only writing the ADT in this question and not the main file you must compile and link your file with the provided main. This can be done in the following commands, assuming the files `maze.c`, `maze.h` and `main.c` are in your current working directory:

```
gcc -Wall -Wvla -Werror main.c
gcc -Wall -Wvla -Werror maze.C
gcc -Wall -Wvla -Werror maze.o main.o -o maze
```

In order to be able to test before you implement every single function, you may want to include empty implementations in your `maze.c` until you are able to finish them. That way you can test the functions you have implemented first by writing test cases that only invoke the functions you have written. For example, if you have written only `readMaze` and `printMaze` you can at least test those two functions by writing a test case that simply provides the maze and then asks to print it. Since you only have a blank implementation for `destroyMaze` in this example your program would leak memory, but you would at least be able to see if you are able to properly read and print a maze. You may if you want even write your own main in another file only for testing, this main must not be submitted.

Deliverables: For this question include in your final submission zip your C implementation file `maze.c`. **NOTE: This file must NOT contain a main function!** This file should contain only the definitions of your `Maze` structure, as well as the definition of all the functions declared in the `maze.h`. It may also include additional helper functions that you wrote to help with your implementation.

3. **Size-Aligned Addresses (Warm-up, not for marks)** Before you begin the next question of this assignment, it may be helpful to perform a warm-up by writing a function which will be very useful to us in the question of the assignment, and also applies the simple concepts that will make the building blocks for that question.

We have discussed in class how data must be stored on *size-aligned* addresses. That is, if one is trying to write data of size *X* to memory then the starting address of the data must be divisible by *X*. For example, with our integer size of 4 on Ohaton we know immediately that an integer could not be stored at the address `0x7af408c2` because that number is *not* divisible by 4, the next closest address (going up in addresses) to that one which would be usable for storing an integer would be `0x7af408c4`.

For this warm-up you should write the function `nextAlignedAddress` which has two parameters, described below.

- `void *faddr` — An arbitrary memory address, given to your function. Semantically this represents the start of free memory, but for the purposes of this question all that matters is that it is a pointer to a memory address. Since this memory does not point at anything in particular (it is just the first address that is free memory) its type is a `void *`.
- `size_t size` — An unsigned integer which represents the size of some data your program wants to write to free memory.

Your function should return a `void *` which is the pointer to the first address after (or at) `faddr` that is size-aligned to `size`. This function should only take a few lines!

Hints: There are some important hints to be made on this question and assignment. Here, you will certainly need to use a tool we've used in class but not discussed in great detail — that is the ability to *cast* data types from one type to another. Primarily in this assignment you will need to treat addresses as pointers to different types at different times. This means you will have to often cast a pointer from one type to another (so that you can read from or write to it). To cast the result of an expression to a certain type you simply use the type in parentheses as a prefix unary operator. So the expression `(T) expr` where *T* is a type will evaluate the expression `expr` and then treat its result as if it were a type *T*. There is an example provided with the assignment named `castingExample.c`.

Also, note that when you have a `void*` type addition with that pointer will add the arguments exactly (since what is the size of the data that a `void *` points at?). So for example if you have the following:

```
void *p = 0x4003;
```

Then `p+3` would simply be `0x4006` as there would be no multiplier for the operand 3 as is typical in pointer arithmetic.

Finally, note that if you want to do more complex arithmetic on a pointer (such a division operations or similar) you cannot do so on a pointer. Instead, you will need to treat that pointer like an unsigned integer type. As such it can be useful to cast pointers to the `size_t` type, which is guaranteed to be large enough to store the pointer.

Deliverables None. This question is a warm-up only. It is not for marks. However, this function will be helpful in the programs to come.

4. Memory Familiarity

In this question we make sure we are comfortable with the concept of memory and understand that a pointer provides us with nothing more than a memory location, and that memory is simply a collection of bytes. For that reason if we are given a memory address we can choose to write *anything* there. In this question we will be storing a series of values

For this question you need to implement two functions: `writeValue` and `printValues`. Both functions are going to work in tandem to treat an allocated block of memory as if it were an array of arbitrary types. In this “array” you need to be able to write any of four types: an integer, a character, a pointer, or a string. Your functions must also take into account the alignment of these types. You are also provided a test harness file which contains the main which will use your two functions — you should make sure to read and understand the test harness so you understand how your functions will be used and how to write test cases for your program.

- `writeValue` is a function which takes three parameters, and whose ultimate purpose is to write a value into memory. However, since we are writing arbitrary values to memory and need to be able to read them back later we need to write to memory not only the *payload* (the actual data asked to be written to memory) but also some *metadata* (data about the payload data).

The parameters of `writeValue` are:

- `void *writeLoc` — the memory address at which you are meant to write the data (and your metadata).
- `const char *type` — A c-string which stores either `"int"`, `"char"`, `"ptr"`, or `"str"`. Which tells you which type you are storing in this particular call.
- `void *payload` — a pointer to the actual data you are being asked to write into memory. It is given as a `void *` since this function must work for writing different types, as such you must treat the `payload` as a pointer to the appropriate type which you determine from the `type` parameter.

The metadata we write can be thought of as a trail of breadcrumbs for a future call to `printValues` to follow, so that it knows when looking at any arbitrary pointer exactly what it is meant to be reading from memory and printing out to the screen.

For our metadata we will use a single byte, so we will treat our metadata as the `char` type. One of the benefits of our metadata being a `char` type is that since on our system `sizeof(char)` is 1 we are able to write our metadata on *any* byte and have it be size-aligned. This way we can always begin the metadata for our next value at whatever address is free after writing the previous value. You may choose any metadata scheme you want, but a simple suggestion is to use the values 0, 1, 2, and 3 each to mean one of the four types you need to store. Then, when looking at a particular address you simply read that first byte to see if it is 0, 1, 2, or 3 to know what data you have.

Below is the procedure `writeValue` must perform.

- (a) First, determine the type you are working with, you should map the `type` string to whatever your metadata constant is for the given type.
- (b) Next, you must write your metadata constant at the given `writeLoc` address, as the type signature should be the first thing found at each “value” stored in your memory.
- (c) Now that you have written to `writeLoc` a character of metadata you must find the next space where it would be appropriate to write your value. The byte directly

at `writeLoc` you have just used, so the next available free byte is `writeLoc + 1`. However, depending on the type you are asked to write `writeLoc+1` may not be an address that is size-aligned with the type you need to store. As such, you should find the immediate next address in which you can write your value and write it there. For example, if `writeLoc` was `0x55d653e1000` and you are asked to write an integer then you wrote your metadata to byte `0x55d653e1000` making the next free byte `0x55d653e1001`. However, that is not an address that is divisible by `sizeof(int)` and as such you must go to the next address that would be. In our system since `sizeof(int)` is 4 that would mean you write the actual payload to the address `0x55d653e1004`, leaving 3 bytes of “wasted” space (which we call padding).

- (d) After writing the payload to the appropriate space you should return the pointer to the next free byte. In the example above the next free byte would be `0x55d653e1008`, since you wrote an integer occupying 4 bytes to the address `0x55d653e1004`.

- `printValues` is a function which takes a single parameter, whose purpose is to simply print out as many values that were written to memory with `writeValue`.

The parameter of `printValues` is:

- `void *dataStart` — a memory address at which `writeValue` was called *or* a memory address where the first byte is initialized to `0xFF` (meaning the byte stores all 1s).

Given this information your function `printValues` must look at the data at `dataStart` and determine if it was a value written with `writeValue`. If it was in fact a value written with `writeValue` then you should use your metadata to determine the type of value written and print out that value following the same format as the sample executable. The function then must repeat this process for the next address in memory after the value you just read, and continue doing so until it finds memory that was not written with `writeValue`.

Below is the procedure `printValues` must perform.

- (a) First, determine if you have a value written by `writeValue`, and if so which type. If `writeValue` is written properly this can be done by reading the byte value of the byte stored at `dataStart`. If the value is `0xFF` then it should not be a value created with `writeValue`, if it is any other value then it should be one of your four type tags — use that type tag for the following steps. When the byte *is* `0xFF` then there is nothing more for your function to do as you have reached the end of memory that was written to by your `writeValue` function.
- (b) When you do have a value written by `writeValue` you should use your type tag to tell you what data you should print. You must then find the address after the type tag where the actual payload is stored, which should be the next size-aligned address after `dataStart`. Once you find the data, you should print it out.
- (c) After printing out the data found, you should repeat the process for the next free byte after the data you printed, in case there is another value written there!

The test harness will read in different writes the user wants to make, and make each of them using your `writeValue` function. Since your `writeValue` function always returns the next free address the return value of `writeValue` is always used as the next argument to be provided as the `writeLoc`. In this way, your function repeatedly writes values to memory in order, no matter the type of the data. After the user finishes providing input then your `printValues` function is called on the original starting address of the data, which should then print out all the values that were read in and stored to memory.

To help with understanding the process below is provided a series of inputs from the user, and a memory diagram that demonstrates how you will fill in memory as you write values. This example assumes you are using the value 0, 1, 2, and 3 to tag the types

`int`, `char`, `ptr`, and `str` respectively. The diagrams below also assume a *little-endian* system, so whenever a data type which comprises multiple bytes is written out in the diagram it is written out from its lowest order byte to its highest order byte.

Initially, every bit in the given memory is initialized by the harness to be all ones, so that wherever you finish writing with `writeValue` will have the next byte recognizable by your `printValue` as not in use. The smallest address in the example `0x7f f7e8` is our initial `writeLoc`. This leaves us with the following memory diagram:

0x7f f808	1111 1111	1111 1111	1111 1111	1111 1111
0x7f f804	1111 1111	1111 1111	1111 1111	1111 1111
0x7f f800	1111 1111	1111 1111	1111 1111	1111 1111
0x7f f7fc	1111 1111	1111 1111	1111 1111	1111 1111
0x7f f7f8	1111 1111	1111 1111	1111 1111	1111 1111
0x7f f7f4	1111 1111	1111 1111	1111 1111	1111 1111
0x7f f7f0	1111 1111	1111 1111	1111 1111	1111 1111
0x7f f7ec	1111 1111	1111 1111	1111 1111	1111 1111
0x7f f7e8	1111 1111	1111 1111	1111 1111	1111 1111

Consider what happens when the user types `int 10`, then in that case we must store the number 10. First, we write the tag 0 to the given `writeLoc` which is `0x7f f7e8`. However, writing that tag means our next free byte would be `0x7f f7e9` which is not an address size-aligned to the size of an integer. So we write the integer itself at the next size-aligned address which is `0x7f f7ec`, and we then return the next free byte as the location `0x7f f7f0`.

0x7f f808	1111 1111	1111 1111	1111 1111	1111 1111
0x7f f804	1111 1111	1111 1111	1111 1111	1111 1111
0x7f f800	1111 1111	1111 1111	1111 1111	1111 1111
0x7f f7fc	1111 1111	1111 1111	1111 1111	1111 1111
0x7f f7f8	1111 1111	1111 1111	1111 1111	1111 1111
0x7f f7f4	1111 1111	1111 1111	1111 1111	1111 1111
0x7f f7f0	1111 1111	1111 1111	1111 1111	1111 1111
0x7f f7ec	int_0: 0000 1010	int_1: 0000 0000	int_2: 0000 0000	int_3: 0000 0000
0x7f f7e8	tag: 0000 0000	1111 1111	1111 1111	1111 1111

In the above diagram the tag byte is prefixed with `tag:`. Similarly, each of the bytes that comprise the integer read in are prefixed with `int_n` where the `n` is which byte of the integer that is written there, 0 being the lowest order byte and 3 being the highest order byte.

Note how only the tag byte was written and then the needed space for the integer was written, and all other memory is left untouched. The bytes between our tag and our

integer are left as is, though since they are padding bytes they are considered now in use by us, so we could have written an extra value there if we found it helpful for debugging.

Consider, next, that the user types `str abcde`. In that case, first the function `writeValue` must write the tag `3` to indicate this is a string, and then begin writing the characters of the string: `'a'`, `'b'`, `'c'`, `'d'`, `'e'`, and `'0'`. In that case our memory digram looks like the following:

0x7f f808	1111 1111	1111 1111	1111 1111	1111 1111
0x7f f804	1111 1111	1111 1111	1111 1111	1111 1111
0x7f f800	1111 1111	1111 1111	1111 1111	1111 1111
0x7f f7fc	1111 1111	1111 1111	1111 1111	1111 1111
0x7f f7f8	1111 1111	1111 1111	1111 1111	1111 1111
0x7f f7f4	char: 'd'	char: 'e'	char: '\0'	1111 1111
0x7f f7f0	tag: 0000 0011	char: 'a'	char: 'b'	char: 'c'
0x7f f7ec	int_0: 0000 1010	int_1: 0000 0000	int_2: 0000 0000	int_3: 0000 0000
0x7f f7e8	tag: 0000 0000	1111 1111	1111 1111	1111 1111

Notice how this time the type tag is the byte that represents the value `3`, because we are storing a string. Also notice how the characters of our string start immediately after the tag with no need for padding, since our characters can be stored on any address as the size of our character is one, meaning any address is size-aligned for a character. Since the last byte of our string (the one storing the null terminator) is at the address `0x7f f7f6` that means our next free address (the value we return) is `0x7f f7f7`.

Imagine next, the user enters `int 256`, then the following update to our memory would take place.

0x7f f808	1111 1111	1111 1111	1111 1111	1111 1111
0x7f f804	1111 1111	1111 1111	1111 1111	1111 1111
0x7f f800	1111 1111	1111 1111	1111 1111	1111 1111
0x7f f7fc	1111 1111	1111 1111	1111 1111	1111 1111
0x7f f7f8	int_0: 0000 0000	int_1: 0000 0001	int_2: 0000 0000	int_3: 0000 0000
0x7f f7f4	char: 'd'	char: 'e'	char: '\0'	tag: 0000 0000
0x7f f7f0	tag: 0000 0011	char: 'a'	char: 'b'	char: 'c'
0x7f f7ec	int_0: 0000 1010	int_1: 0000 0000	int_2: 0000 0000	int_3: 0000 0000
0x7f f7e8	tag: 0000 0000	1111 1111	1111 1111	1111 1111

Here, we have a nice result! Once again, since our type tags are only one byte long we can write them at any address. In this case that means we write the type tag at the current free address `0x7f f7f7` then we find the next free size-aligned address for our integer. In this case that happens to be the immediate next byte `0x7f f7f8`, so we don't have to waste any space at all! This example is important to note what the behaviour of your `writeValue` function should be. It only skips over space *when necessary*.

While the above example demonstrates a little-endian machine you do not need to take the endianness of your machine into account should you write your program properly. That is, when you want to write an integer you simply find the appropriate address to do so, treat that address as a pointer to an integer, and write the given integer to that address. Your system will write that integer at that address in whatever layout is appropriate for your system.

Note: Don't be fooled... While we say in this question we are building an “array”, since we are laying out values in contiguous memory addresses, what we are actually achieving is more of a linked-list since we require linear lookup time to find the *n*th item. Instead of pointers to the next element we have our type tags which tell us how far to look for the next element after this one, we still need to walk each of these type tags to get to the *n*th item.

Deliverables For this question include in your final submission zip your C source code file named `writeval.c`. You do not need to (nor should you) include `writeval.h` or `main.c` as we will simply replace them with the original copies that were given to you. So make sure you do not change either of `writeval.h` or `main.c` to make your solution work, as those changes will get overwritten when you submit.

How to submit: Create a zip file `a2.zip`, make sure that zip file contains your C source code files `sequences.c`, `maze.c`, and `writeval.c`. Assuming all three of these files are in your current working directory you can create your zip file with the command

```
$ zip a3.zip sequences.c maze.c writeval.c
```

Upload your file `a3.zip` to the a3 submission link on Canvas.